

# A Drift-Aware MLOps Framework for Multi-Model Serving: A Network Intrusion Detection Case Study

Hoang Tran-Nguyen-Viet<sup>1</sup>, Thai Bui-Ngoc<sup>1</sup>, Tuan Le-Anh<sup>1</sup>, Bao Nguyen-Van<sup>1</sup>,  
and Quan Le-Trung<sup>1</sup>

Faculty of Computer Networks and Communications,  
University of Information Technology, VNU-HCM,  
Ho Chi Minh City, Vietnam  
{23520541,23521412}@gm.uit.edu.vn, {tuanla,baonv,quanlt}@uit.edu.vn

**Abstract.** Machine learning services can degrade when production data distributions change, even if source code remains unchanged. This issue is critical for network intrusion detection, where benign traffic and attack behavior evolve over time. This paper presents a baseline drift-aware MLOps framework for user-imported and training-job-created tabular models, using Network Intrusion Detection Systems (NIDS) as a case study rather than proposing a new classifier. The contributions are threefold: a tenant-aware control-plane/data-plane architecture in which a PostgreSQL-backed Native Registry owns product-level lifecycle state; a drift-aware formulation connecting onboarding, inference logs, reference and current windows, feature decisions, dataset drift, and human-reviewed actions; and prototype feasibility evidence from a controlled NIDS challenger evaluation and a bounded serving replay. The configured prototype combines Django/FastAPI services, Redpanda, Argo and Kubeflow resources, S3-compatible storage, optional MLflow support, and Evidently AI. The evidence does not constitute production-scale validation.

**Keywords:** MLOps · Data Drift · Multi-model Serving · Model Monitoring · Network Intrusion Detection

## 1 Introduction

Machine learning (ML) models are increasingly deployed as continuously running services rather than offline experimental artifacts. In a production setting, a model receives new input samples over time, generates predictions for downstream users or systems, and depends on monitoring mechanisms to remain trustworthy. Unlike traditional software services, an ML service may degrade even when its source code and infrastructure remain unchanged. One major cause is Data Drift, where the production data distribution differs from the reference distribution used during training, validation, or initial deployment.

This problem is especially relevant in Network Intrusion Detection Systems (NIDS). Network traffic is non-stationary: benign usage patterns change, infrastructure evolves, and attack strategies continuously adapt. A classifier trained on historical flow features may therefore become less reliable when deployed in a new environment or under a new attack distribution. An operational NIDS should not only expose a prediction API but also record serving data, compare it with reference data, and initiate review when the monitored distribution changes.

Existing MLOps research provides architectural concepts for automating ML development, deployment, and monitoring [1]. Other works discuss MLOps maturity, pipeline platforms, data quality, and continual model maintenance [4, 7, 8]. Within the tool families reviewed in Section 2, tool-centered deployments can leave drift reports outside the tenant-facing lifecycle state used for serving and review decisions.

This paper presents a baseline framework that connects multi-model serving with drift-aware monitoring. The goal is to describe a reusable MLOps framework, not to claim a new state-of-the-art NIDS classifier. In the proposed design, NIDS is used as a case study because it naturally exposes the need for drift-aware operation. The framework is intentionally scoped to tabular classification models that are either imported by users or produced by controlled training jobs and then converted into serving-compatible artifacts.

The main contributions of this paper are:

- We propose a tenant-aware control-plane/data-plane architecture in which the Native Registry owns product lifecycle state in PostgreSQL while serving and tracking tools act as adapters.
- We formalize a drift-aware lifecycle using tenants, deployable model records, reference and current windows, feature-level decisions, dataset-level drift ratio, and human-reviewed actions.
- We provide prototype feasibility evidence through a NIDS case study, a controlled offline challenger evaluation, and a bounded serving replay.

The remainder of this article is structured as follows. Section 2 reviews related work on MLOps lifecycles, serving platforms, data quality, drift monitoring, and ML-based NIDS. Sections 3–5 define the problem, present the framework, and describe the drift-aware lifecycle algorithm; Sections 6–8 instantiate the framework in the NIDS case study, describe the prototype, and report preliminary evaluation results. Sections 9 and 10 discuss limitations and conclude the paper.

## 2 Related Work

MLOps extends DevOps principles to ML systems by coordinating data engineering, experimentation, training, deployment, monitoring, and governance. Kreuzberger *et al.* define MLOps as a paradigm for reliably and efficiently maintaining ML systems in production [1]; John *et al.* propose an adoption maturity model [4]; Sculley *et al.* identify technical debt beyond model code, including

**Table 1.** MLOps tool-family scope and remaining gap.

| Tool family        | Main support                                                           | Gap relative to this work                                                                         |
|--------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| MLflow             | Tracking, artifacts, registry.                                         | Tenant ownership, lifecycle state, serving logs, and UI actions need product control-plane logic. |
| Flow tools         | Workflow orchestration.                                                | Registry governance, multi-model serving, and drift-driven review remain external concerns.       |
| Kubeflow/TFX       | Pipeline and cloud-native ML workflows.                                | Tenant lifecycle, registry views, and drift summaries must be mapped into application metadata.   |
| Proposed framework | Registry, serving, logs, drift, and artifacts under one control plane. | Baseline prototype; production hardening and autonomous promotion remain future work.             |

data dependencies, configuration, monitoring, and feedback loops [2]; and Amer-shi *et al.* show that ML engineering differs from traditional software engineering because data, models, and code evolve together [3]. This paper narrows these lifecycle concerns to drift-aware signals for tenant-aware multi-model serving.

Platform studies provide complementary building blocks. TFX integrates ingestion, validation, training, and serving [6]; MLflow supports tracking and model lifecycle management [12]; and Clipper frames online prediction as a systems problem [13]. Pipeline and domain case studies [5, 7], self-adaptive MLOps architecture work [18], and a recent empirical comparison of MLflow, Metaflow, Airflow, and Kubeflow Pipelines [20] show complementary strengths rather than one universal lifecycle abstraction.

Table 1 relates these reviewed tool families to the lifecycle-state gap addressed here.

Production ML reliability depends on post-deployment data as well as the pipelines that manage it. Renggli *et al.* treat data quality as a first-class MLOps concern [8], and Polyzotis *et al.* identify production data-management challenges [9]. Data Drift and concept drift are related but distinct: Gama *et al.* survey adaptation to changing streams [10], while Webb *et al.* characterize forms of concept drift [11]. Here, Data Drift is an operational signal: because labels may be delayed or unavailable, the system first compares input distributions and then triggers inspection or workflow handoff.

Network intrusion detection is a non-stationary case study for this lifecycle. CIC-IDS2017 provides labeled intrusion traffic [15], but real environments can differ from dataset conditions. Sommer and Paxson highlight deployment context, generalization, and evolving traffic challenges for ML-based intrusion detection [16], and recent NIDS work treats concept drift as an operational problem requiring adaptation policies [19]. This paper uses NIDS to illustrate

lifecycle and monitoring requirements, not to claim a new state-of-the-art intrusion detection classifier.

For the tool families reviewed above, tracking, orchestration, pipeline execution, and serving remain useful building blocks, but their use in tool-centered deployments does not by itself define tenant-facing lifecycle-state ownership. This paper addresses that specific gap through a Native Registry that owns tenant, model-family, version, deployability, log-summary, drift-result, and review state in the control plane, while the reviewed tools remain adapters.

### 3 Problem Formulation

Let  $\mathcal{U} = \{u_1, u_2, \dots, u_N\}$  denote the set of tenants using the platform. Each tenant  $u \in \mathcal{U}$  owns a set of registered models:

$$\mathcal{M}_u = \{m_{u,1}, m_{u,2}, \dots, m_{u,K_u}\}. \quad (1)$$

Each model  $m_{u,i}$  is represented by an inference function parameterized by  $\theta$ :

$$\hat{y}_t = f_{\theta}^{u,i}(x_t), \quad (2)$$

where  $x_t \in \mathbb{R}^d$  is the input vector at time  $t$ ,  $d$  is the number of input features, and  $\hat{y}_t$  is the prediction. For a tabular classification model,  $\hat{y}_t$  may be a class label or a probability vector over class labels.

For every inference request, the platform records a production log:

$$z_t = (u, i, x_t, \hat{y}_t, c_t, t), \quad (3)$$

where  $c_t$  is the prediction confidence if available. Here,  $m_{u,i}$  denotes one registered deployable model record (a version within a tenant-owned model family), and the log stream associated with  $(u, i)$  forms the basis for monitoring.

Each monitored model record has a reference dataset:

$$D_{\text{ref}}^{u,i} = \{x_1, x_2, \dots, x_n\}, \quad (4)$$

which represents the distribution observed during training, validation, or initial acceptance. At monitoring time  $T$ , the current production data window is:

$$D_{\text{cur}}^{u,i}(T) = \{x_t \mid T - W \leq t \leq T\}, \quad (5)$$

where  $W$  is the monitoring window size. The objective is not only to compute predictions but also to select a lifecycle action:

$$a_T = \pi(m_{u,i}, D_{\text{ref}}^{u,i}, D_{\text{cur}}^{u,i}(T), S_T), \quad (6)$$

where  $\pi(\cdot)$  is the adaptation policy and  $S_T$  is the lifecycle state owned by that deployable model record. Possible actions include no operation, alert generation,

manual review, retraining review triggers, and candidate rollback or promotion records for human review.

For each feature  $j$ , the framework computes a drift score:

$$\delta_j = \Delta(P_{\text{ref}}(x_j), P_{\text{cur}}(x_j)), \quad (7)$$

where  $\Delta(\cdot)$  returns a feature-level test statistic or distance between the reference and current distributions. For a hypothesis test, let  $p_j$  be its p-value;  $\delta_j$  remains the statistic used to describe effect magnitude. The decision indicator is:

$$I_j = \begin{cases} 1, & p_j < \alpha, & \text{for a hypothesis-test rule,} \\ 1, & \delta_j > \tau_j, & \text{for a distance rule,} \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

where  $\alpha$  is the test significance level and  $\tau_j$  is a distance threshold when that rule is used. The dataset-level drift ratio is:

$$\rho = \frac{1}{d} \sum_{j=1}^d I_j. \quad (9)$$

The final drift decision is:

$$\text{Drift}(D_{\text{ref}}, D_{\text{cur}}) = \begin{cases} 1, & \rho \geq \tau_D, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

where  $\tau_D$  is the dataset-level threshold. This formulation separates model prediction from monitoring and makes Data Drift an explicit input to lifecycle decisions.

## 4 Proposed Framework

The logical flow of the proposed framework consists of four stages. First, a user onboards a model artifact or registers an artifact produced by a controlled training workflow through the control plane. Second, the data plane exposes the model as a prediction endpoint. Third, each prediction produces an asynchronous log event. Fourth, the monitoring layer periodically reconstructs the current production window and compares it with the reference data. If drift is detected, the adaptation layer creates an alert or triggers a retraining review workflow.

### 4.1 Control Plane

The control plane manages user-facing and governance operations. It stores tenants, authentication credentials, API keys, model metadata, artifact locations, endpoint identifiers, access modes, and model statuses. In the submitted prototype, this layer uses Django. The PostgreSQL-backed Native Registry is the

product-level source of truth for model ownership, versions, deployability metadata, log summaries, drift results, and lifecycle state.

When a model artifact is uploaded or registered from a completed training job, the control plane validates the available metadata, stores artifact references, and records the model in the Native Registry. The record can then be assigned an endpoint identifier of the form `/models/{model_id}/predict`; this metadata alone does not assert that a dedicated runtime endpoint has been deployed.

## 4.2 Data Plane

The submitted data plane uses a generic FastAPI model server. The logical path resolves model metadata through a shared API, while deployment adapters may create dedicated runtime resources when configured. Artifact loading is adapter-based: MLflow pyfunc is an optional internal adapter where configured and does not own product lifecycle state or require user training scripts to import MLflow.

The serving design resolves tenant credentials and model ownership before inference. Public records can be invoked directly, while private records require an API key or bearer token and an owner match. This tenant-aware policy supports independent model records in a shared serving layer; security-strength validation is outside the reported evaluation.

## 4.3 Production Logging Layer

Online inference should not depend synchronously on database writes. Therefore, the framework decouples prediction from logging through a message broker. After producing a prediction, the model server emits a production log event to Redpanda. A consumer persists events to PostgreSQL. Each log stores the tenant and model identifiers, timestamp, feature payload, prediction, and optional confidence  $c_t$ .

This design has two advantages. First, it keeps monitoring writes outside the synchronous prediction path. Second, it creates a unified production data source that can be queried per tenant and per model for drift detection, audit, and evaluation.

## 4.4 Drift Monitoring Layer

The monitoring layer compares reference data with recent production data. In the prototype, Evidently AI is used to compute feature-level drift and dataset-level drift summaries. Reference data can be stored as object storage artifacts, while production data are reconstructed from PostgreSQL logs. Drift reports can be exported as HTML or JSON artifacts for inspection.

## 4.5 Adaptation Layer

The adaptation layer maps monitoring outputs to lifecycle actions. In the baseline implementation, the primary action is a drift alert or workflow trigger. The

**Require:** Model package or training artifact  $A$ , metadata  $H$ , tenant  $u$   
**Ensure:** Registered model record with endpoint/deployability metadata  $e$

- 1: Validate package extension and archive structure
- 2: Validate serving metadata or record deployability status
- 3: Store or reference  $A$  in artifact storage
- 4: Create model record  $m_{u,i}$  in the control plane
- 5: Assign lifecycle state  $S \leftarrow \text{validated}$
- 6: Record endpoint identifier  $e \leftarrow \text{/models/\{i\}/predict}$
- 7: Return model record and  $e$

**Fig. 1.** Model onboarding procedure.

**Require:** Input  $x_t$ , model identifier  $i$ , credential  $q$   
**Ensure:** Prediction  $\hat{y}_t$  and optional lifecycle action  $a_T$

- 1: Authenticate  $q$  and resolve tenant  $u$
- 2: Retrieve model metadata  $m_{u,i}$
- 3: Load or reuse cached model  $f_{\theta}^{u,i}$
- 4: Compute prediction  $\hat{y}_t \leftarrow f_{\theta}^{u,i}(x_t)$
- 5: Emit log event  $z_t = (u, i, x_t, \hat{y}_t, c_t, t)$ , with optional  $c_t$
- 6: **if** monitoring condition is satisfied **then**
- 7:   Build production window  $D_{\text{cur}}^{u,i}(T)$
- 8:   **for** each feature  $j = 1$  to  $d$  **do**
- 9:     Compute drift score  $\delta_j$
- 10:    Compute indicator  $I_j$
- 11:   **end for**
- 12:   Compute drift ratio  $\rho$
- 13:   Select action  $a_T \leftarrow \pi(m_{u,i}, D_{\text{ref}}, D_{\text{cur}}, S_T)$
- 14:   **if**  $\rho \geq \tau_D$  **then**
- 15:     Trigger alert, review, or workflow handoff
- 16:   **end if**
- 17: **end if**
- 18: Return  $\hat{y}_t$

**Fig. 2.** Drift-aware inference and monitoring procedure.

framework can be extended to support retraining jobs, model evaluation gates, review-driven promotion, rollback, and notification workflows. This distinction is important: the current prototype demonstrates a drift-aware control loop, while fully autonomous retraining and deployment remain future work.

Figures 1 and 2 summarize the framework-level onboarding and drift-aware monitoring procedures.

## 5 Drift-Aware Lifecycle Algorithm

The model lifecycle can be represented as a framework-level state vocabulary attached to each registered deployable model record:

$$S \in \{\text{uploaded, validated, deployed, monitored, drifted, reviewing, retraining, promoted, failed}\}. \quad (11)$$

The transition function is:

$$S_{T+1} = \Phi(S_T, m_{u,i}, \rho, \tau_D, a_T), \quad (12)$$

where  $\Phi(\cdot)$  updates the record state according to the drift ratio and selected action. If  $\rho < \tau_D$ , the record remains monitored; otherwise it may be marked drifted and handed to alerting or review. The retraining, promotion, and rollback vocabulary does not imply autonomous execution: such actions remain human-reviewed or future-work paths in this baseline.

## 6 Case Study: Network Intrusion Detection

The NIDS model is used as a case study to instantiate the proposed framework. A network flow is represented as a tabular feature vector  $x \in \mathbb{R}^d$  containing statistics such as duration, packet counts, byte rates, and protocol-related attributes. The model predicts whether the observed traffic is benign or belongs to an attack class.

In the current prototype, XGBoost is selected as the baseline classifier rather than as a claim of state-of-the-art NIDS performance. The case study uses tabular network-flow features, where tree-based gradient boosting provides a practical balance between non-linear feature interaction modeling, training cost, inference efficiency, and artifact portability [14]. Compared with simple linear baselines, XGBoost can capture interactions among flow statistics such as duration, packet counts, and byte rates. Compared with deep neural models, it requires less task-specific architecture design and is easier to package, version, and serve as a compact artifact in a multi-tenant MLOps prototype. Therefore, XGBoost is used to exercise the lifecycle—training, packaging, registration, endpoint invocation, production logging, and drift monitoring—rather than to optimize NIDS classification accuracy. Other classifiers can be onboarded through the same artifact and registry workflow when they satisfy the serving contract.

NIDS is an appropriate case study for drift-aware MLOps for three reasons. First, network environments change over time, causing benign traffic distributions to shift. Second, attack patterns evolve, making historical training data incomplete. Third, labeled production data are often delayed or unavailable, so input distribution monitoring can provide an early warning signal before supervised performance metrics are available.

## 7 Prototype Implementation

The prototype uses a control-plane/data-plane split: the Django/DRF control plane records tenant-facing lifecycle state in PostgreSQL, while the FastAPI

data plane resolves model records, invokes loader adapters, and emits log events. Table 2 summarizes prototype modules, adapters, and configured responsibilities.

Figure 3 summarizes the configured prototype/testbed workflow across control, serving, training, storage, logging, and monitoring components.

**Table 2.** Prototype modules, adapters, and configured responsibilities.

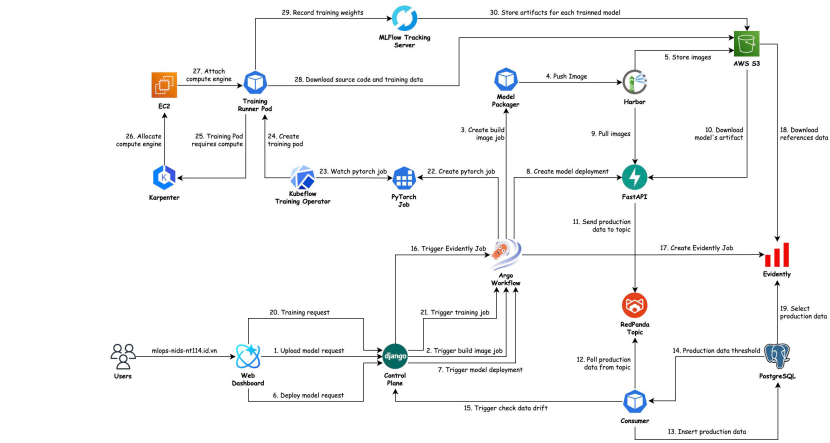
| Module                   | Implementation                  | Responsibility                                                |
|--------------------------|---------------------------------|---------------------------------------------------------------|
| Control plane            | Django/DRF                      | Records tenants, jobs, registry, deployment, and drift state. |
| Training execution       | Argo/Kubeflow + runner          | Configures entry-point execution and artifact return.         |
| Artifact/registry layer  | Object storage + PostgreSQL     | Stores artifacts; owns product metadata and version state.    |
| Packaging/serving        | Packager + FastAPI              | Configures artifact preparation, loading, and endpoint paths. |
| Logging/drift            | Redpanda + consumer + Evidently | Configures log persistence and drift summaries.               |
| Deployment configuration | Docker Compose/K8s/Argo files   | Supports local integration and prototype/testbed deployment.  |

Figures 4 and 5 provide interface evidence for recorded registry state and an illustrative monitoring report.

*Training execution.* In the configured Kubernetes path, the control plane submits a training webhook to Argo Events, and Argo Workflows may instantiate a Kubeflow `PyTorchJob` on Karpenter-backed nodes where configured. The runner path executes the selected entry point, exchanges `model.tar.gz` through S3-compatible storage, and returns completion metadata. User scripts need not import MLflow; optional tracking metadata does not determine Native Registry validity.

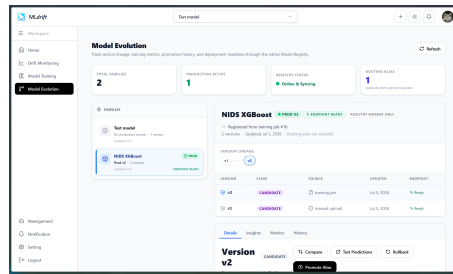
*Metadata and registry.* On job-artifact ingestion, the control plane records available metrics, parameters, insights, and deployability summaries. The Native Registry groups family and version records linked to `ModelAPI` records, with PostgreSQL remaining the product-level source of truth for tenant ownership, lifecycle, and drift results. MLflow, when configured, provides internal tracking or serving adaptation rather than product-state ownership.

*Packaging, serving, and monitoring.* The configured packaging path can build an image through Argo/Docker, push it to Harbor, and request a FastAPI pod; Section 8 does not evaluate each branch. In the prototype flow, prediction logs pass asynchronously through Redpanda to PostgreSQL, and Evidently produces monitoring reports. Drift remains a review signal, not an automatic redeployment decision.

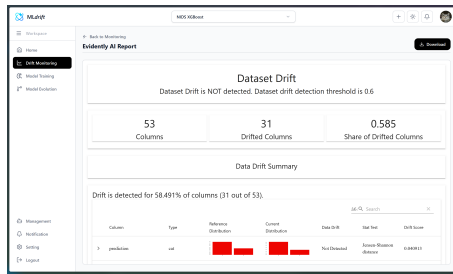


**Fig. 3.** Configured prototype/testbed workflow. The Django control plane owns registry and lifecycle metadata; the FastAPI data plane handles serving and log emission. Argo/Kubeflow, object storage, Harbor, MLflow, Evidently, and Karpenter-backed compute are supporting adapters where configured.

*Deployment and operational scope.* The submitted testbed configuration includes Docker Compose, Kubernetes (K3s) manifests, and Argo templates. Node provisioning through Karpenter is configured but its elasticity is not evaluated. The evidence exercises a controlled offline evaluation and one bounded serving replay; autonomous promotion, canary deployment, and production hardening remain future work.



**Fig. 4.** Model Evolution UI recording v1–v2 lineage and registry-recorded serving-readiness metadata.



**Fig. 5.** Illustrative drift-report UI from a separate prototype run, not one of the nine controlled Table 3 windows.

## 8 Preliminary Evaluation

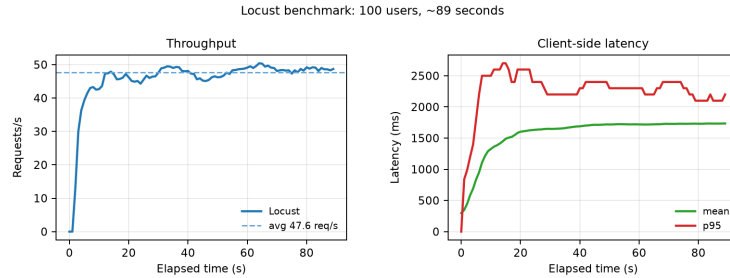
Evaluation focuses on feasibility under one controlled split and run (seed 42), not NIDS optimization. The evaluator reads local tables `train_2_classes.csv`, `reference_data.csv`, and three attack-family drift tables, referred to here as CIC-IDS2017-style data because the stored artifacts do not document their complete derivation from the original release. Each table provides 52 numeric flow features plus `Label`; non-BENIGN labels map to ATTACK, nonnumeric values are coerced, and missing or infinite values are imputed with medians from the fixed-model training sample. The fixed XGBoost model uses at most 100,000 sampled training rows after exact future-ATTACK row hashes are removed.

Each attack table is shuffled and split into 60% trigger/retraining and 40% future-holdout pools. Trigger/future counts are 30,000 and 20,000 for PortScan, 4,890 and 3,260 for BruteForce, and 1,029 and 686 for WebAttacks. A per-family challenger uses the cleaned fixed-model training rows plus its trigger ATTACK rows. For this run,  $D_{\text{ref}}$  is the 23,873 BENIGN rows in the seed-42, 50,000-row sample of `reference_data.csv`; held-out windows combine up to 5,000 future ATTACK rows with BENIGN rows from that same pool. Mild/Moderate/Severe window sizes are 28,873/12,500/5,882, 21,733/8,150/3,835, and 4,573/1,714/807 for the three families, respectively. BENIGN rows are therefore not disjoint between  $D_{\text{ref}}$  and the controlled windows. Hashing numeric features plus normalized labels removes exact future-ATTACK duplicates from model fitting; it does not exclude session-, capture-, preprocessing-, correlated-flow-, or semantic overlap.

For each of 52 features, a two-sided KS test sets  $I_j = 1$  at uncorrected  $p_j < \alpha = 0.05$ ; no multiple-testing correction is applied. The dataset rule is  $\rho \geq \tau_D$  with  $\tau_D = 0.30$ . The reported mean KS is the mean test statistic and is descriptive, not the alert rule. No repeated runs, confidence intervals, or statistical comparisons were performed. Table 3 reports the single-run results.

**Table 3.** Controlled single-run challenger evaluation. Mean KS is descriptive across 52 features;  $\Delta\text{M-F1}$  is computed before display rounding.

| Attack Family | Level    | Atk% | Mean KS | Fixed M-F1 | Chall. M-F1 | $\Delta\text{M-F1}$ |
|---------------|----------|------|---------|------------|-------------|---------------------|
| PortScan      | Mild     | 17%  | 0.101   | 0.453      | 0.999       | +0.547              |
|               | Moderate | 40%  | 0.234   | 0.375      | 1.000       | +0.625              |
|               | Severe   | 85%  | 0.493   | 0.130      | 1.000       | +0.869              |
| BruteForce    | Mild     | 15%  | 0.068   | 0.460      | 1.000       | +0.540              |
|               | Moderate | 40%  | 0.181   | 0.375      | 1.000       | +0.625              |
|               | Severe   | 85%  | 0.386   | 0.131      | 1.000       | +0.869              |
| WebAttacks    | Mild     | 15%  | 0.085   | 0.460      | 0.997       | +0.538              |
|               | Moderate | 40%  | 0.224   | 0.375      | 0.996       | +0.621              |
|               | Severe   | 85%  | 0.482   | 0.130      | 0.983       | +0.853              |



**Fig. 6.** One bounded testbed replay: 100 Locust users over an  $\approx 89$ -second window.

All nine windows satisfy the 0.30 dataset rule, with 50–51 of 52 features marked drifted in the stored results. As the controlled attack fraction rises, mean KS increases from 0.07–0.10 to 0.39–0.49 and fixed-model Macro-F1 falls from about 0.46 to 0.13. The corresponding challengers achieve Macro-F1 of 0.983–1.000. These near-perfect values may reflect highly separable selected attack patterns and the constructed BENIGN/ATTACK mixtures. They demonstrate lifecycle feasibility only in this controlled setting, not robustness across environments, natural drift, or general adaptation effectiveness.

One bounded testbed replay used 100 Locust users for approximately 89 seconds and issued 4,343 client-side requests. Locust recorded 47.6 requests/s on average, 1.74 s mean response time, 2.30 s p95, and four HTTP 502 responses. Prometheus recorded a 144.1 requests/s peak server-side rate and 1.60 s mean handler latency. Client averages and server-side peaks use different measurement and aggregation semantics and are not directly comparable. This single observation is not capacity, scalability, reliability, availability, SLA, or production evidence. Figure 6 shows the client-side time series.

## 9 Discussion

The framework treats monitoring reports as control-plane lifecycle evidence. The Native Registry matters as the tenant-facing owner of model-version, deployability, log-summary, drift-result, and review state while serving and monitoring remain replaceable adapters.

The scope is intentionally limited. First, the system targets tabular classification artifacts; image, text, time-series, and large language models need additional schema validation and monitoring metrics. Second, arbitrary user-uploaded artifacts introduce security risks, so hardened systems must isolate execution, validate dependencies, restrict archive extraction, and enforce resource quotas. Third, autonomous retraining and promotion require reliable evaluation gates, rollback policies, and human approval. Therefore, this baseline treats drift detection as an adaptation signal rather than an unconditional redeployment decision.

## 10 Conclusion

This paper presented a baseline drift-aware MLOps framework with a tenant-aware control-plane/data-plane architecture, a drift-aware lifecycle formulation, and preliminary NIDS case-study evidence. Its PostgreSQL-backed Native Registry remains the product-level source of truth for ownership, versions, deployability metadata, log summaries, drift results, and lifecycle state; MLflow and other tools are adapters where configured. One controlled challenger run and one bounded testbed replay support prototype feasibility, not general effectiveness or production-scale validation. NIDS remains a case study, and retraining, promotion, or rollback require human review rather than autonomous lifecycle management.

## References

1. M. Kreuzberger, N. Kuhl, and S. Hirschl, “Machine Learning Operations (MLOps): Overview, Definition, and Architecture,” *IEEE Access*, vol. 11, pp. 31866–31879, 2023.
2. D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden Technical Debt in Machine Learning Systems,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
3. S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, “Software Engineering for Machine Learning: A Case Study,” in *Proc. IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, 2019, pp. 291–300.
4. M. M. John, H. H. Olsson, and J. Bosch, “Towards MLOps: A Framework and Maturity Model,” in *Proc. 47th Euromicro Conference on Software Engineering and Advanced Applications*, 2021, pp. 1–8.
5. Y. Zhou, Y. Yu, and B. Ding, “Towards MLOps: A Case Study of ML Pipeline Platform,” in *Proc. International Conference on Artificial Intelligence and Computer Engineering*, 2020, pp. 494–500.
6. D. Baylor et al., “TFX: A TensorFlow-Based Production-Scale Machine Learning Platform,” in *Proc. 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, pp. 1387–1395.
7. S. G. Subramanya, S. Shetty, V. Patil, S. K. Gad, and S. N. Bhat, “From DevOps to MLOps: Overview and Application to Electricity Market Forecasting,” *Applied Sciences*, vol. 12, no. 19, 2022.
8. C. Renggli, B. Karlaš, B. Ding, F. Liu, K. Schawinski, W. Wu, and C. Zhang, “A Data Quality-Driven View of MLOps,” *arXiv preprint arXiv:2102.07750*, 2021.
9. N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data Management Challenges in Production Machine Learning,” in *Proc. ACM International Conference on Management of Data*, 2017, pp. 1723–1726.
10. J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A Survey on Concept Drift Adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.
11. G. I. Webb, R. Hyde, H. Cao, H.-L. Nguyen, and F. Petitjean, “Characterizing Concept Drift,” *Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 964–994, 2016.

12. M. Zaharia et al., “Accelerating the Machine Learning Lifecycle with MLflow,” *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.
13. D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, “Clipper: A Low-Latency Online Prediction Serving System,” in *Proc. 14th USENIX Symposium on Networked Systems Design and Implementation*, 2017, pp. 613–627.
14. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
15. I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proc. 4th International Conference on Information Systems Security and Privacy*, 2018, pp. 108–116.
16. R. Sommer and V. Paxson, “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
17. B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.
18. H. Bhatt, S. Biswas, S. Rakhunathan, and K. Vaidhyanathan, “HarmonE: A Self-Adaptive Approach to Architecting Sustainable MLOps,” arXiv preprint arXiv:2505.13693, 2025. Accepted to ECSA 2025. DOI: 10.48550/arXiv.2505.13693.
19. X. Zhang, R. Zhao, Z. Jiang, H. Chen, Y. Ding, E. C. H. Ngai, and S.-H. Yang, “Continual Learning with Strategic Selection and Forgetting for Network Intrusion Detection,” arXiv preprint arXiv:2412.16264, 2025. Accepted by IEEE INFOCOM 2025. DOI: 10.48550/arXiv.2412.16264.
20. J. Marcos-Mercadé, U. Lopez-Novoa, and M. Egaña Aranguren, “An Empirical Evaluation of Modern MLOps Frameworks,” arXiv preprint arXiv:2601.20415, 2026.